

Tyler Buck
CS499
July 17th, 2026

Enhancement One Narrative

Artifact Description

The artifact selected for this enhancement is **Weigh to Go**, an Android mobile application created in **CS 360: Mobile Architecture and Programming** during C-2 (Mar-Apr). The application allows users to create an account, log in, set a goal weight, record and manage daily weight entries, and receive an SMS notification when a recorded weight matches the goal. Account information, goals, weight records, and phone numbers are stored in a local SQLite database.

The original application demonstrated Android development skills through multiple Activities, input validation, SQLite operations, and SMS integration. However, its Activities contained a mixture of interface code, validation, business logic, and direct database access. This tight coupling made the application more difficult to maintain, test, and extend.

Selection and Enhancement

I selected this artifact because it allowed me to demonstrate software engineering skills beyond creating a functional application. Its authentication, goal-management, weight-tracking, and SMS workflows provided several opportunities to improve the internal design while preserving the user-facing behavior.

The primary enhancement was refactoring the application into a simplified Model-View-ViewModel-style architecture. I created a centralized `Repository` class that manages database operations through the existing `DatabaseHelper`. I also created four ViewModel-style classes:

- `LoginViewModel`
- `DashboardViewModel`
- `AddWeightViewModel`
- `SmsSettingsViewModel`

These classes now handle validation, business decisions, and repository operations. The Activities remain responsible for interface interactions, navigation, permission handling, SMS delivery, and displaying results to the user.

The enhancement removed direct database access from the active Activities and established a clearer flow:

Activity → ViewModel → Repository → DatabaseHelper

This structure reduces coupling, improves readability, and makes future changes easier to isolate. The application's original functionality—including authentication, goal management, adding, editing, and deleting weights, data persistence, and SMS alerts—was preserved and tested after the refactor.

Course Outcome Alignment

This enhancement met the primary outcome identified in my Module One plan:

Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.

The MVVM-style refactor demonstrates this outcome by applying separation of concerns, layered architecture, centralized data access, validation, and incremental testing to an existing Android application. The enhanced design is easier to understand, maintain, and extend than the original Activity-heavy implementation.

The enhancement also made progress toward the security outcome by centralizing validation and reducing inconsistent data handling. However, passwords are still stored as plain text in the local SQLite database. Secure password storage was outside the approved scope of this enhancement, but identifying this limitation demonstrates security awareness and establishes a future improvement opportunity.

I did not substantially change my original outcome-coverage plan. The completed work remained focused on software engineering and design, with supporting evidence of security awareness. Other outcomes will be demonstrated more fully through the remaining artifact enhancements and the final professional self-assessment. The rubric permits each milestone to demonstrate progress toward selected outcomes rather than every program outcome at once.

Reflection

This enhancement reinforced that improving software does not always require adding features. The application already worked, but its internal structure made future maintenance more difficult. Refactoring it required preserving its behavior while moving responsibilities into more appropriate layers.

One challenge was deciding how much architectural complexity to introduce. I considered a complete Android Jetpack MVVM implementation, but that approach would have expanded the project beyond the approved scope. I instead chose a simplified MVVM-style structure that

demonstrated separation of concerns without unnecessarily redesigning the entire application. This taught me that good engineering involves selecting a solution appropriate to the project rather than automatically choosing the most complex option.

Another challenge was avoiding regressions while removing database logic from several Activities. I addressed this by refactoring one workflow at a time: authentication, dashboard operations, weight entry, and SMS settings. After each change, I built and tested the related feature before continuing. This incremental process helped me locate issues such as leftover database references, initialization order problems, missing repository methods, and misplaced braces.

Through this work, I strengthened my understanding of architectural design, refactoring, validation, debugging, and maintainable Android development. I also learned that successful enhancement work depends on balancing technical improvement, project scope, and preservation of existing functionality.